Índice

🏢 **SecureCorp — Identidad y Cifrado**

- Qué vas a hacer
  - Cómo está escrita
  - Cómo organizar el tiempo
- Parte 0 · El laboratorio
  - La primera vez
  - Las órdenes del kit

BLOQUE A · Certificados y LDAP

- A1 · La autoridad de certificación (CA)
- A2 · El certificado del servidor LDAP
- A3 · El directorio de SecureCorp
- A4 · LDAPS: que el directorio solo hable cifrado

BLOQUE B · Kerberos

- B1 · Crear el reino
- B2 · Usuarios y servicio
- B3 · Tickets

PARTE C · Por dentro del laboratorio: Dockerfile y docker-compose

- Mini-reto (opcional, +0,5) · Marta se va de vacaciones
- Entrega

🏢 SecureCorp — Identidad y Cifrado

Qué vas a hacer

Eres el administrador/a de sistemas de **SecureCorp** (`securecorp.local`). En dos bloques:

- **Bloque A:** crear los **certificados** de la empresa y un **directorio LDAP** con sus empleados que solo hable **cifrado**.
- **Bloque B:** crear el **reino Kerberos** y conseguir **tickets**.
- **Parte C:** ver cómo está hecho el laboratorio por dentro y añadirle **tú** una máquina con un `Dockerfile` y el `docker-compose.yml`. Lo necesitarás en la próxima práctica.

Cómo está escrita

Cada apartado tiene estas partes:

Lo que tienes que saber	La idea, en pocas líneas
Ejemplo	Un caso resuelto: ejecútalo y fíjate en qué hace
Tu turno	Algo parecido que tienes que hacer **tú**, con pistas
Cómo sabes que está bien	El comando para comprobarlo y lo que tiene que salir

> **Tu mejor amigo: `./lab.sh comprobar`**
>
> Ejecútalo (en tu ordenador) al terminar cada apartado: te dice en **verde** lo que está bien y en **rojo** lo que falta.

Cómo organizar el tiempo

La práctica dura **unas 4 horas y media**. Hazla en **dos sesiones** y no corras: lo importante es entender cada paso, porque en la defensa te lo voy a preguntar.

Sesión	Qué haces	Tiempo aproximado
1	Parte 0 (laboratorio) + A1 + A2 + A3	2 h
2	A4 + Bloque B + Parte C + mini-reto (si te da tiempo)	2 h
En casa o al final	Contestar `respuestas.md` y hacer la entrega	30 min

Al terminar la sesión 1 apaga el laboratorio con `./lab.sh parar`. En la sesión 2 lo enciendes con `./lab.sh arrancar` y está todo como lo dejaste.

Parte 0 · El laboratorio

No tienes que instalar nada ni saber Docker. El kit trae un script, **`lab.sh`**, que construye y enciende las máquinas por ti. Solo necesitas **Internet la primera vez** que haces `preparar`, porque descarga lo que hace falta.

La primera vez

Descarga el kit de Moodle, abre una terminal en la carpeta donde lo hayas guardado y:

```
unzip kit-sad-p1-securecorp.zip      # descomprime el kit
cd sad-p1-securecorp                 # entra en su carpeta
chmod +x lab.sh                      # da permiso de ejecución al script
./lab.sh preparar                    # tu nombre y tu usuario + construir las máquinas (unos minutos)
./lab.sh arrancar                    # encender las máquinas
```

Las órdenes del kit

Todas se escriben **en tu ordenador**, dentro de la carpeta `sad-p1-securecorp`:

Orden	Para qué
`./lab.sh arrancar`	Encender las máquinas (cada día, al empezar)
`./lab.sh servidor`	Entrar en el **servidor** (prompt **rojo**)
`./lab.sh cliente`	Entrar en el **cliente** (prompt **verde**)
`./lab.sh reiniciar`	Reiniciar el servidor después de cambiar su configuración. No borra nada
`./lab.sh rescate`	Si el servidor no arranca: entrar a arreglarlo
`./lab.sh logs kerberos`	Ver en directo lo que hace el servidor Kerberos (Ctrl+C para salir)
`./lab.sh comprobar`	Ver qué llevas bien y qué te falta
`./lab.sh entregar`	Generar el .zip de la entrega
`./lab.sh parar`	Apagar las máquinas al terminar (no se pierde nada)
`./lab.sh reset`	Borrar las máquinas y empezar de cero (tus certificados y LDIF se conservan)
`./lab.sh ayuda`	Ver esta lista

Tu usuario = inicial del nombre + primer apellido, en minúsculas y sin tildes (María Núñez → `mnunez`). En este enunciado aparece como **`TUUSUARIO`**.

Trabajas en **tres sitios**. Mira siempre el prompt:

Dónde	Cómo llegar	Prompt
Tu ordenador	La terminal normal, en la carpeta del kit	`tu@tupc:~/sad-p1-securecorp$`
SERVIDOR (LDAP + Kerberos)	`./lab.sh servidor`	**root@SERVIDOR**
CLIENTE (PC de un empleado)	`./lab.sh cliente`	**root@CLIENTE**

Para volver a tu ordenador: `exit`. Te recomiendo **tres pestañas** en la terminal.

Dato	Valor
Administrador LDAP	`cn=admin,dc=securecorp,dc=local` · contraseña **SecureCorp2026**
Servidor LDAP	`ldap.securecorp.local`
Reino Kerberos	`SECURECORP.LOCAL` (siempre en mayúsculas)
Las carpetas `pki/` y `ldif/` del kit	Dentro de las máquinas son `/pki` y `/ldif`

BLOQUE A · Certificados y LDAP

A1 · La autoridad de certificación (CA)

Lo que tienes que saber. La CA es «el notario» de la empresa: firma los certificados de los servidores, y los equipos que confían en la CA confían en lo que ella firma. Un certificado tiene un `subject` (de quién es) y un `issuer` (quién lo ha firmado).

Ejemplo: tu CA. Créala en el servidor:

```
# SERVIDOR
```

```
mkdir -p /pki/ca && cd /pki/ca

# La clave privada de la CA: el "sello" del notario
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out ca.key

# El certificado de la CA, firmado con su propia clave. ⚠️ Pon tu nombre en el CN
openssl req -x509 -new -key ca.key -sha256 -days 3650 \
  -subj "/C=ES/O=SecureCorp/CN=SecureCorp Root CA - Tu Nombre Apellido" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" \
  -out ca.crt
```

Tu turno. Con `openssl x509 -in ca.crt -noout` seguido de una o varias de estas opciones, averigua lo que te pregunta `respuestas.md` (pregunta 1):

Opción	Qué enseña
`-subject`	De quién es el certificado
`-issuer`	Quién lo ha firmado
`-dates`	Desde cuándo y hasta cuándo es válido
`-text`	Todo

`./lab.sh comprobar` → **A1** en verde.

A2 · El certificado del servidor LDAP

Lo que tienes que saber. Para que el servidor tenga su certificado se hacen tres pasos:

> **Pasos:** 1 · Clave privada — ldap.key → 2 · Petición (CSR) — ldap.csr — «soy ldap.securecorp.local» → 3 · La CA la firma — ldap.crt
>
> La CA firma con **ca.crt + ca.key**.

Ejemplo. El paso 1 es igual que la clave de la CA, pero más corta:

```
# SERVIDOR
mkdir -p /pki/servidor && cd /pki/servidor
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out ldap.key
```

Y estas son las **extensiones** que llevará el certificado. Créalas tal cual:

```
cat > ldap.ext <<'EOF'
basicConstraints = CA:FALSE
keyUsage = critical, digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = DNS:ldap.securecorp.local
EOF
```

Tu turno. Completa los `___` de los pasos 2 y 3 y ejecútalos:

```
# 2 · La petición. El CN es el NOMBRE con el que los clientes llaman al servidor
openssl req -new -key ___ -subj "/C=ES/O=SecureCorp/CN=___" -out ldap.csr

# 3 · La CA firma la petición (necesita el certificado Y la clave de la CA)
openssl x509 -req -in ___ -CA ___ -CAkey ___ \
  -CAcreateserial -days 825 -sha256 -extfile ldap.ext -out ldap.crt
```

> **Pista**
>
> Los ficheros de la CA están en /pki/ca/ y los del servidor en /pki/servidor/ (donde estás). Si te equivocas, repite los pasos 2 y 3: machacan lo anterior.

Cómo sabes que está bien:

```
openssl verify -CAfile /pki/ca/ca.crt ldap.crt       # → ldap.crt: OK
openssl x509 -in ldap.crt -noout -subject -issuer    # issuer = tu CA
```

`./lab.sh comprobar` → **A2** en verde.

A3 · El directorio de SecureCorp

Lo que tienes que saber. El directorio es un árbol. Cada entrada tiene un **DN** (su dirección completa) y unos atributos. Se cargan con ficheros **LDIF**. Al terminar, el árbol de SecureCorp será así:

```
dc=securecorp,dc=local
├── ou=people
│   ├── uid=lromero — Lucía · ejemplo
│   ├── uid=TUUSUARIO — tú · lo creas tú
│   └── uid=mtorres — Marta · lo creas tú
└── ou=groups
    ├── cn=rrhh — Lucía y Marta · lo creas tú
    └── cn=sistemas — tú · lo creas tú
```

Ejemplo: Lucía. Abre `ldif/01-ejemplo.ldif` y léelo. Luego cárgalo, ponle contraseña y comprueba que puede entrar:

```
# CLIENTE
cd /ldif
ldapadd -x -H ldap://ldap.securecorp.local -D "cn=admin,dc=securecorp,dc=local" -W -f 01-ejemplo.ldif
#   (-W te pide la contraseña del admin: SecureCorp2026)

ldappasswd -x -H ldap://ldap.securecorp.local -D "cn=admin,dc=securecorp,dc=local" -W \
  -s 'Lucia2026' "uid=lromero,ou=people,dc=securecorp,dc=local"

ldapwhoami -x -H ldap://ldap.securecorp.local -D "uid=lromero,ou=people,dc=securecorp,dc=local" -w
'Lucia2026'
#   → dn:uid=lromero,ou=people,dc=securecorp,dc=local

# Buscar: el uid de todas las personas
ldapsearch -x -LLL -H ldap://ldap.securecorp.local -b "ou=people,dc=securecorp,dc=local"
"(objectClass=inetOrgPerson)" uid
```

Y así es **un grupo** (este es inventado, no lo cargues):

```
dn: cn=direccion,ou=groups,dc=securecorp,dc=local
objectClass: groupOfNames
cn: direccion
member: uid=jefa,ou=people,dc=securecorp,dc=local
member: uid=jefe,ou=people,dc=securecorp,dc=local
```

Tu turno.

1. Escribe en `ldif/02-tu-turno.ldif` **4 entradas**: **tú** (uidNumber 10001), **Marta Torres** (`mtorres`, 10003), el grupo **`rrhh`** (Lucía y Marta) y el grupo **`sistemas`** (tú). Las instrucciones están dentro del fichero.

2. Cárgalo con `ldapadd`, igual que el ejemplo.
3. Pon contraseña a **ti** y a **Marta** con `ldappasswd`, igual que a Lucía. Apúntalas en `respuestas.md`.
4. Escribe y ejecuta estas dos búsquedas, y pega el resultado en `respuestas.md` (pregunta 2):
   - **a)** los `member` del grupo `rrhh` (pista: busca en `ou=groups` con el filtro `"(cn=rrhh)"`);
   - **b)** el `cn` y el `mail` de todas las personas.

Si ldapadd da error

Error	Qué significa
Already exists (68)	Esa entrada ya estaba cargada. Las de **después** no se cargan: bórralas del fichero o añade `-c` a `ldapadd` para que siga
requires attribute 'sn' (u otro)	Te falta un atributo obligatorio: compara con Lucía
Invalid syntax (21) / invalid format	Falta una línea en blanco, o un atributo está mal escrito
No such object (32)	Lo de arriba en el árbol no existe: ¿cargaste antes 01-ejemplo.ldif?

`ldapwhoami` funciona contigo y con Marta, y `./lab.sh comprobar` → **A3** en verde.

A4 · LDAPS: que el directorio solo hable cifrado

Lo que tienes que saber. Ahora mismo todo, incluidas las contraseñas, viaja **en claro** por el puerto **389**. Para cifrarlo con TLS (puerto **636**, `ldaps://`) hacen falta cuatro cosas:

#	Qué	Dónde
1	Los certificados en un sitio que slapd pueda leer	Servidor
2	Decirle a slapd dónde están	Servidor (`cn=config`)
3	Que escuche en 636 y deje de escuchar en 389	Servidor (`/etc/default/slapd`)
4	Que el cliente confíe en tu CA	Cliente (`/etc/ldap/ldap.conf`)

1 · Los certificados. Cópialos:

```
# SERVIDOR
mkdir -p /etc/ldap/tls
cp /pki/ca/ca.crt /pki/servidor/ldap.crt /pki/servidor/ldap.key /etc/ldap/tls/
```

slapd funciona con el usuario y grupo **`openldap`**, no como root. Haz que **todo** `/etc/ldap/tls` sea de `openldap` y que la clave `ldap.key` **solo la pueda leer y escribir su dueño**.

Pista

`chown -R usuario:grupo carpeta` cambia el dueño de una carpeta y todo lo que hay dentro.

`chmod` con números: `6` = leer y escribir, `0` = nada; el orden es dueño, grupo, otros.

`ls -l /etc/ldap/tls` → los tres ficheros de `openldap` y la clave con `-rw-------`.

2 · Decirle a slapd dónde están. Completa las tres rutas de `ldif/03-tls.ldif` y aplícalo:

```
# SERVIDOR
ldapmodify -Y EXTERNAL -H ldapi:/// -f /ldif/03-tls.ldif     # → modifying entry "cn=config"
```

Si sale `Other (e.g., implementation specific) error (80)`: una ruta está mal o slapd no puede leer el fichero (revisa el paso 1).

3 · Abrir 636 y cerrar 389. slapd escucha en lo que diga la línea `SLAPD_SERVICES` de `/etc/default/slapd`:

Valor	Qué es
`ldap:///`	Puerto 389, en claro
`ldaps:///`	Puerto 636, cifrado
`ldapi:///`	Socket local, solo desde el propio servidor (lo usas con `-H ldapi:///`). **No lo quites**

Edita con `nano /etc/default/slapd` la línea `SLAPD_SERVICES="..."` para que quede **solo lo que necesitas**. Luego sal (`exit`) y reinicia **desde tu ordenador**:

```
./lab.sh reiniciar
```

> **Si dice que el servidor NO arranca**
>
> Lee el error que enseña (suele ser una errata). Arréglalo con `./lab.sh rescate`, que te deja entrar aunque el servidor esté parado, sal con `exit` y vuelve a hacer `./lab.sh reiniciar`.

4 · Que el cliente confíe en tu CA. Prueba desde el cliente:

```
# CLIENTE
ldapwhoami -x -H ldaps://ldap.securecorp.local -D "uid=lromero,ou=people,dc=securecorp,dc=local" -w
'Lucia2026'
#   → Can't contact LDAP server (-1)
```

Ese error no dice mucho. Con `-d 1` se ve el motivo real:

```
ldapwhoami -d 1 -x -H ldaps://ldap.securecorp.local -D "uid=lromero,ou=people,dc=securecorp,dc=local" -w
'Lucia2026' 2>&1 | grep TLS
#   → TLS: peer cert untrusted   (el cliente no conoce a tu CA)
```

Añade al final de `/etc/ldap/ldap.conf` **del cliente** una línea `TLS_CACERT` seguida de la ruta del certificado de tu CA (pista: el cliente también tiene `/pki`).

Cómo sabes que está bien:

```
# CLIENTE
ldapwhoami -x -H ldaps://ldap.securecorp.local -D "uid=lromero,ou=people,dc=securecorp,dc=local" -w
'Lucia2026'   # ✔ funciona
ldapwhoami -x -H ldap://ldap.securecorp.local  -D "uid=lromero,ou=people,dc=securecorp,dc=local" -w
'Lucia2026'   # ✘ ya no
```

`./lab.sh comprobar` → **A4** en verde.

BLOQUE B · Kerberos

Lo que tienes que saber. Recuerda el parque temático: con tu contraseña consigues una **pulsera** (TGT) en la taquilla (**AS**); enseñando la pulsera pides en el mostrador (**TGS**) el **ticket** de cada atracción (servicio). Tu contraseña **no viaja** por la red.

> **Mira el log del KDC mientras trabajas**
>
> En una pestaña de **tu ordenador**: `./lab.sh logs kerberos` (Ctrl+C para salir). Cada pulsera sale como `AS_REQ` y cada ticket como `TGS_REQ`.

B1 · Crear el reino

```
# SERVIDOR
kdb5_util create -s -r SECURECORP.LOCAL     # te pide una "master key": invéntala y apúntala
service krb5-kdc start
```

B2 · Usuarios y servicio

Ejemplo: Lucía y el servicio web.

```
# SERVIDOR
kadmin.local -q "addprinc lromero"                          # pon la misma contraseña que en LDAP
kadmin.local -q "addprinc -randkey host/web.securecorp.local"   # un servicio: clave aleatoria
kadmin.local -q "listprincs"
```

Tu turno. Crea los principals de **ti** y de **Marta**, con la misma contraseña que en LDAP.

`listprincs` muestra a los tres y al servicio.

B3 · Tickets

Ejemplo: Lucía consigue su pulsera.

```
# CLIENTE
kinit lromero       # te pide su contraseña
klist           # su pulsera: krbtgt/SECURECORP.LOCAL@SECURECORP.LOCAL
kdestroy            # la tira (como cerrar sesión)
```

Tu turno. Con **tu** usuario: consigue tu pulsera y después un ticket para el servicio `host/web.securecorp.local`. Para pedir un ticket de un servicio se usa `kvno <servicio>`. Antes de hacer `kdestroy`, copia la salida de `klist` en `respuestas.md` (pregunta 6).

`klist` enseña **dos** tickets (la pulsera y el de la web), y en el log ves tu `AS_REQ` y tu `TGS_REQ`. `./lab.sh comprobar` → **B** en verde.

PARTE C · Por dentro del laboratorio: Dockerfile y docker-compose

Lo que tienes que saber. Hasta ahora `./lab.sh` te ha escondido Docker. En la próxima práctica no habrá `lab.sh`, así que vamos a ver qué hay debajo. Son dos ficheros:

Fichero	Qué es	Parecido a…
`Dockerfile`	La **receta** de una máquina: de qué sistema parte, qué se instala y qué hace al encender	El guion para instalar un PC
`docker-compose.yml`	El **plano** del laboratorio: qué máquinas hay, en qué red están y qué carpetas comparten	El esquema de la red del aula

Con la receta se **construye** una imagen (`docker compose build`) y con la imagen se **enciende** una máquina, un contenedor (`docker compose up -d`). `./lab.sh preparar` y `./lab.sh arrancar` hacían eso por ti.

Ejemplo: así es el cliente. Abre `docker-compose.yml` con tu editor y busca este trozo:

```
cliente:                        # nombre del servicio (para docker compose)
  image: sad-p1-securecorp          # imagen que usa (la construye el servicio servidor)
  container_name: sad-p1-cliente    # nombre del contenedor (para docker exec)
  hostname: cliente                 # nombre de la máquina (sale en el prompt)
  volumes:
    - ./pki:/pki                    # carpeta de TU ordenador : carpeta DENTRO de la máquina
    - ./krb5/krb5.conf:/etc/krb5.conf:ro    # :ro = solo lectura
  networks:
    - securecorp                    # la red donde están todas las máquinas
  depends_on:
    - servidor                      # no arranca hasta que arranque el servidor
```

Y el `Dockerfile` (en la misma carpeta) es la receta: `FROM` (de qué parte), `RUN` (qué ejecuta al construir: instalar paquetes), `COPY` (qué ficheros mete) y `CMD` (qué hace al encender). Ábrelo y léelo: lo vas a necesitar.

Prueba estas órdenes **en tu ordenador**, en la carpeta del kit:

```
docker compose ps                   # qué máquinas están encendidas
docker compose config --services    # qué servicios hay en el docker-compose.yml
docker compose exec cliente bash    # entrar en el cliente (es lo que hace ./lab.sh cliente)
```

Tu turno: la máquina web de SecureCorp. En el bloque B creaste el servicio `host/web.securecorp.local`, pero esa máquina no existe. Créala tú.

1 · La receta. Completa los `____` de `web/Dockerfile` (las instrucciones están dentro): parte de la misma Debian que el laboratorio, instala el cliente de Kerberos y el de LDAP, confía en tu CA y se queda encendida.

2 · El plano. Primero haz una copia por si lo rompes:

```
cp docker-compose.yml docker-compose.yml.bak
```

Busca la línea `# Parte C: aquí va el servicio web` y debajo añade el servicio `web`, completando los `____`:

```
  web:
    build: ____                     # la carpeta donde está su Dockerfile
    container_name: sad-p1-web
    hostname: web
    volumes:
      - ./pki:/pki:ro
      - ./krb5/krb5.conf:____:ro        # ¿dónde lo pone el cliente?
    networks:
      securecorp:
        aliases:
          - ____                    # el nombre con el que la conocen las demás máquinas
    depends_on:
      - ____
```

> ⚠️ **En YAML los espacios importan**
>
> `web:` va con **2 espacios**, igual que `cliente:`, y lo de dentro con 4. Nunca uses tabuladores. Para comprobar que el fichero está bien: `docker compose config --services` tiene que listar `web`.

3 · Constrúyela, enciéndela y entra:

```
docker compose build web          # construir la imagen con tu receta
docker compose up -d web          # encender la máquina
docker compose ps              # ¿aparece sad-p1-web?
docker compose exec web bash       # entrar en ella
```

Dentro de `web`, consigue tu pulsera con `kinit TUUSUARIO` y comprueba que llegas a LDAP **cifrado** con `ldapwhoami -x -H ldaps://ldap.securecorp.local -D "uid=TUUSUARIO,ou=people,dc=securecorp,dc=local" -W`. Esta vez **no** has tenido que tocar `/etc/ldap/ldap.conf` a mano: ya venía en la receta.

Si algo falla

Síntoma	Qué pasa
did not find expected key / yaml: line ...	Mal sangrado en el docker-compose.yml. Compara con cliente, o vuelve a la copia: cp docker-compose.yml.bak docker-compose.yml
sad-p1-web no sale en docker compose ps, y en docker compose ps -a sale Exited (0)	La máquina se enciende y se apaga: ¿has puesto el CMD?
failed to solve al construir	Error en una línea del Dockerfile: lee qué línea dice. ¿Nombre de paquete mal escrito?
Cambias el Dockerfile y no notas nada	Hay que volver a construir: docker compose up -d --build web
kinit habla de ATHENA.MIT.EDU o dice Cannot find KDC	La máquina no ve tu krb5.conf: falta el volumen o la ruta de dentro está mal

`./lab.sh comprobar` → **Parte C** en verde.

Entrega

1. Contesta **`respuestas.md`** (está en la carpeta del kit).
2. En tu ordenador: `./lab.sh entregar` → genera `entrega/SAD-P1-TUUSUARIO.zip`.
3. Sube **ese .zip** a Moodle.

> **No borres el laboratorio después de entregar**
>
> La defensa se hace en tu ordenador, con tu laboratorio funcionando. No hagas `./lab.sh reset`.